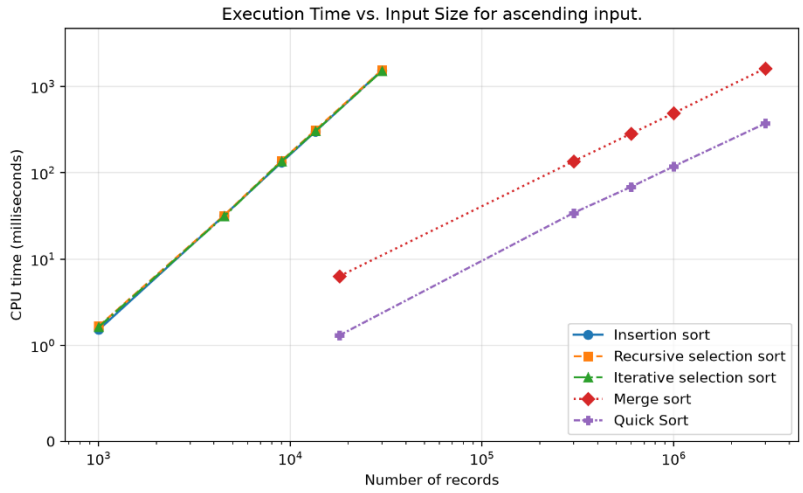
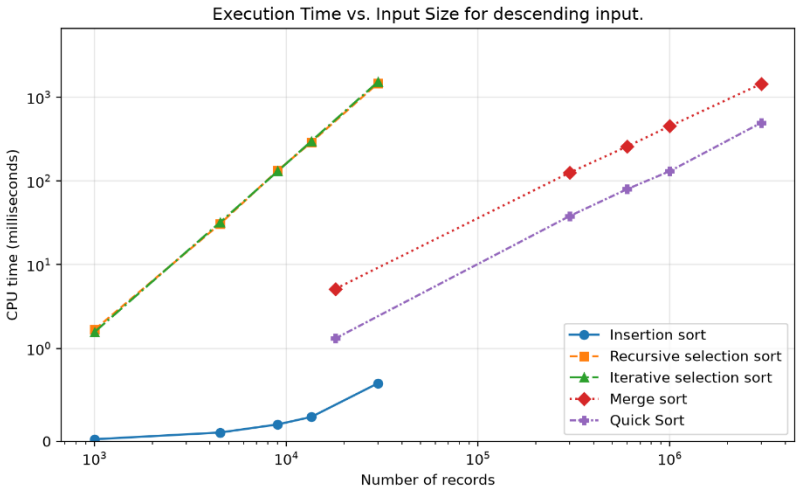
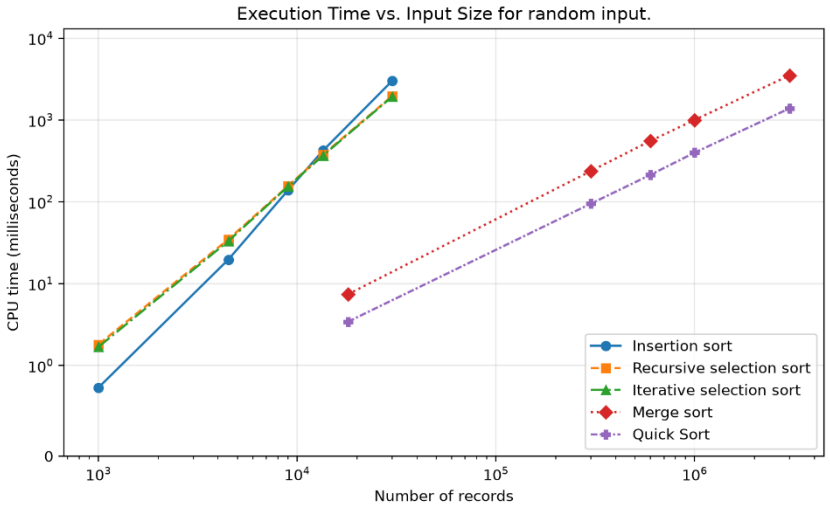






A) For the random input tests, sorting algorithm runtimes vary due to their time complexity. Insertion and selection sort increase significantly as the input size increases, aligning with the $O(n^2)$ time complexity of these algorithms. Note that plots in this report are on log-log axes, linearizing the data visualization. Recursive and iterative selection sort both show similar execution times across all tested sizes. This is expected, since the main difference between the two is only recursion vs. iteration. Merge sort and quick sort show significant improvement over the other sorting algorithms, since their time complexity is $O(n \log n)$. This is because they repeatedly divide the list into smaller sections instead of searching through the unsorted sections, reducing the number of operations required for large inputs. Quick sort consistently outperforms the merge sort algorithm in testing, since it is implemented via a library that is optimized.

B) When the list is already in descending order, insertion sort separates itself from the rest, with a measured execution time significantly less than the other sorting algorithms. This is because our implementation of insertion sort can simply call llist_insert_sorted, which makes its time complexity in this case O(n). Recursive and iterative selection sort show little difference between ordered and random input lists since they have to search through the entire list no matter what. They retain their $O(n^2)$ time complexity. The time complexity of merge and quick sort also does not change when the input is sorted, however, they do show slight decrease in running time.